

1. Введение

Цель документа

Документ описывает концепцию мобильного приложения для автоматизации процесса приемки товаров на складе, включая бизнес-процесс, архитектуру решения, модель данных, интеграцию с учетной системой и пользовательский интерфейс.

Бизнес-проблема

В текущем процессе приемка товаров выполняется вручную. Сотрудник склада сверяет поступивший товар с документом “Поступление товаров”, самостоятельно идентифицирует товар, пересчитывает количество и вручную заносит фактические данные в учетную систему.

Такой подход приводит к следующим проблемам:

- длительное время обработки поставок;
- ошибки при идентификации товаров со схожей упаковкой;
- ошибки при подсчете количества;
- ручной ввод данных в учетную систему;
- риск появления расхождений между фактическими и плановыми данными.

Цель решения

- Ускорить приемку;
- Уменьшить количество ошибок;
- Автоматизировать обмен с учетной системой.

2. Общее описание решения

Предлагается разработать мобильное приложение, которое получает документы поставки из учетной системы посредством REST API, выполняет локальную обработку результатов приемки и передает итоговые данные обратно в учетную систему.

- Учетная система;
- REST API;
- Мобильное приложение;
- Локальное хранилище.

3. Бизнес-процесс

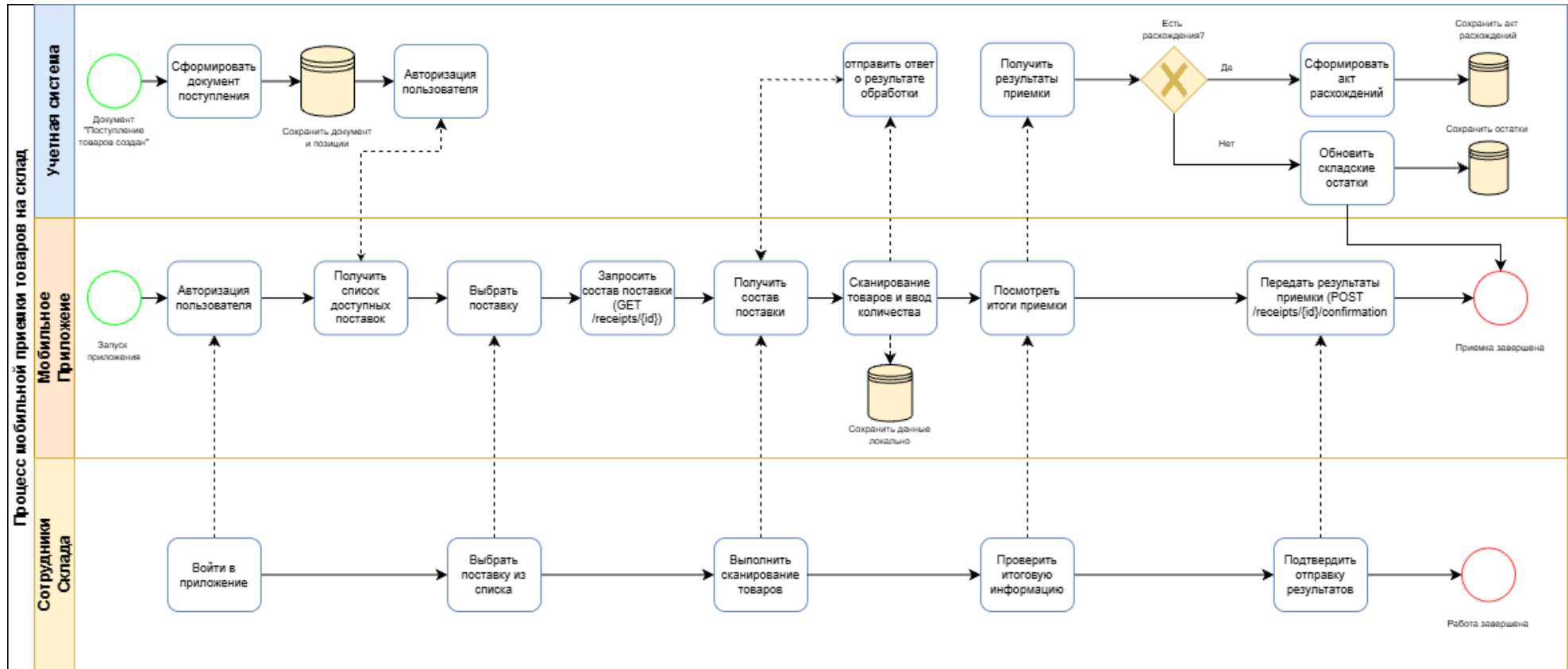
BPMN - это единый графический стандарт для моделирования бизнес-процессов. Он нужен, чтобы перевести сложные регламенты компании на понятный язык для бизнеса и ИТ, исключая двоякое чтение. Схема упрощает поиск узких мест, обучение сотрудников и автоматизацию систем.

Диаграмма наглядно показывает **пошаговый ход процесса**:

- **Кто** выполняет работу (отделы, сотрудники или ИТ-системы).
- **Что** именно делается (конкретные задачи и действия).
- **Как** развивается сценарий при разных условиях (разветвления и шлюзы).
- **Что** запускает и завершает процесс (триггеры и финальные события).

Создание BPMN основывается на **методологии процессного управления и строгой логике алгоритмов**. Процесс раскладывают на базовые элементы: события, задачи, шлюзы ветвления и дорожки ответственности. Моделирование всегда опирается на реальные бизнес-сценарии “как есть” (As-Is) или “как должно быть” (To-Be).

BPMN



Ссылка на Репозиторий с данной схемой - <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Описание BPMN

- **Верхняя дорожка** - Бэкенд / Серверная система (ERP/WMS, где хранятся документы и остатки).
- **Средняя дорожка** - Логика приложения / API (системные действия, запросы и обработка данных).
- **Нижняя дорожка** - Действия пользователя (интерфейс оператора/кладовщика).

Процесс условно делится на 5 основных этапов:

1. Авторизация и инициализация

- **Пользователь:** Нажимает “Войти в приложение”
- **Приложение:** Выполняет “Авторизация пользователя” и отправляет данные на сервер
- **Сервер:** Подтверждает авторизацию и параллельно выполняет действие “Сформировать документ поступления”, сохраняет в БД.

2. Выбор поставки

- **Приложение:** Вызывает функцию “Получить список доступных поставок” с сервера.
- **Пользователь:** Видит данный список на экране и выполняет действие “Выбрать поставку из списка.”
- **Приложение:** Фиксирует “Выбрать поставку” и отправляет на сервер запрос GET /receipts/{id} “Запрос состав поставки”.

3. Сканирование и ввод данных (Основной рабочий блок)

- **Приложение:** Через шаг “Получить состав поставки” открывает форму для работы.
- **Пользователь:** Выполняет действие “Выполнить сканирование товаров”.
- **Приложение:** Переходит в режим “Сканирование товаров и ввод кол-ва”. В этот момент данные отправляются в бекенд и сохраняются локально на устройстве

4. Проверка итогов

- **Пользователь:** Нажимает “Подтвердить отправку результатов”.
- **Приложение:** Отправляет финальный запрос на сервер: POST /receipts/{id}/confirmation “Передать результат приемы”.

5. Завершение и ветвление (Обработка расхождения)

- **Пользователь:** Нажимает “Подтвердить отправку”
- **Приложение:** Отправляет финальный запрос на сервер: POST /receipts/{id}/confirmation “Передать результат приемки”.
- **Сервер:** Запускает шлюз “Есть расхождения?”
 - **ДА:** “Сформировать расхождение.
 - **Нет:** “Обновить складские остатки” в системе.

4. Пользовательские сценарии (User Stories)

User Story 1

Как сотрудник склада,
я хочу видеть список поставок,
чтобы быстро выбрать нужную приемку.

User Story 2

Как сотрудник склада,
я хочу сканировать товар,
чтобы приложение автоматически увеличивало принятое количество.

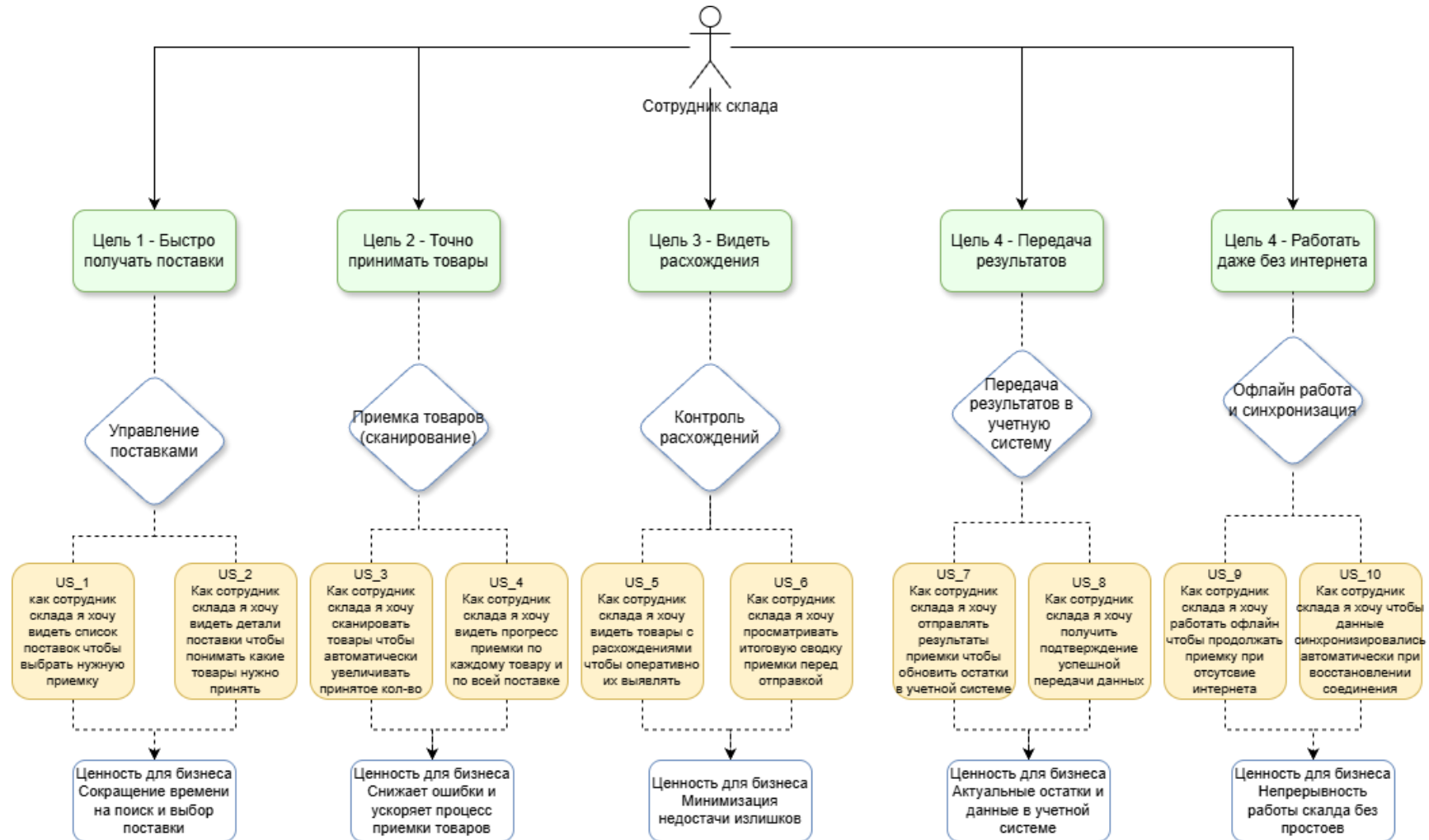
User Story 3

Как сотрудник склада,
я хочу видеть расхождения,
чтобы оперативно обнаруживать ошибки.

User Story 4

Как сотрудник склада,
я хочу отправить результаты,
чтобы учетная система автоматически обновила остатки.

User stories diagram



5. Архитектурные решения

Теперь мы начинаем говорить про техническую реализацию, а для этого составим архитектуру нашего приложения и продумаем каждый блок.

В архитектуре будет UI, Scanner Module, Sync Service, Local Database.

Схема должна наглядно иллюстрировать паттерн Offline-First. Приложение должно выполнять сканирование и сохранять результаты локально без постоянного обращения к серверу, а отправлять накопленные данные в учетную систему только при финальном POST запросе.

Архитектурная схема

Теперь отобразим архитектуру приложения:



Ссылка на Репозиторий с данной схемой - <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Технология локального хранения данных в мобильном приложении (SQLite, CoreData, Room..) не является частью архитектурного решения

1. Слой пользователя (Клиент)

Отвечает за физическое взаимодействие и ввод данных.

- **Сотрудник склада:** Конечный пользователь, который взаимодействует с интерфейсом.
- **Камера / ТСД:** Аппаратный уровень (сканер штрихкодов).

2. Слой мобильного приложения (Фронт)

Построен по модульному принципу и обеспечивает автономность работы на складе. Состоит из 4 внутренних компонентов:

- **UI:** Визуальные экраны пользователя.
- **Scanner Modul:** Программный модуль, который обрабатывает считанные штрихкоды и осуществляет быстрый поиск товаров.
- **Sync service:** Сервис синхронизации, отвечает за отправку и получение данных из учетной системы по HTTPS.
- **Local Storage:** Локальная БД на устройстве. Хранит документы, Справочники товаров и промежуточные результаты приемки. Позволяет работать офлайн.

3. Слой учетной системы (Бек)

Центральный сервер (WMS/ERP) где сосредоточена основная бизнес-логика и мастер-данные.

- **Бизнес-логика:** Модули управления поступлениями, обновления остатков, автоматического формирования актов расхождений и управления справочником.
- **Данные (БД):** Хранилище данных – товаров, остатков, документов поступлений, актов расходов и других справочников.

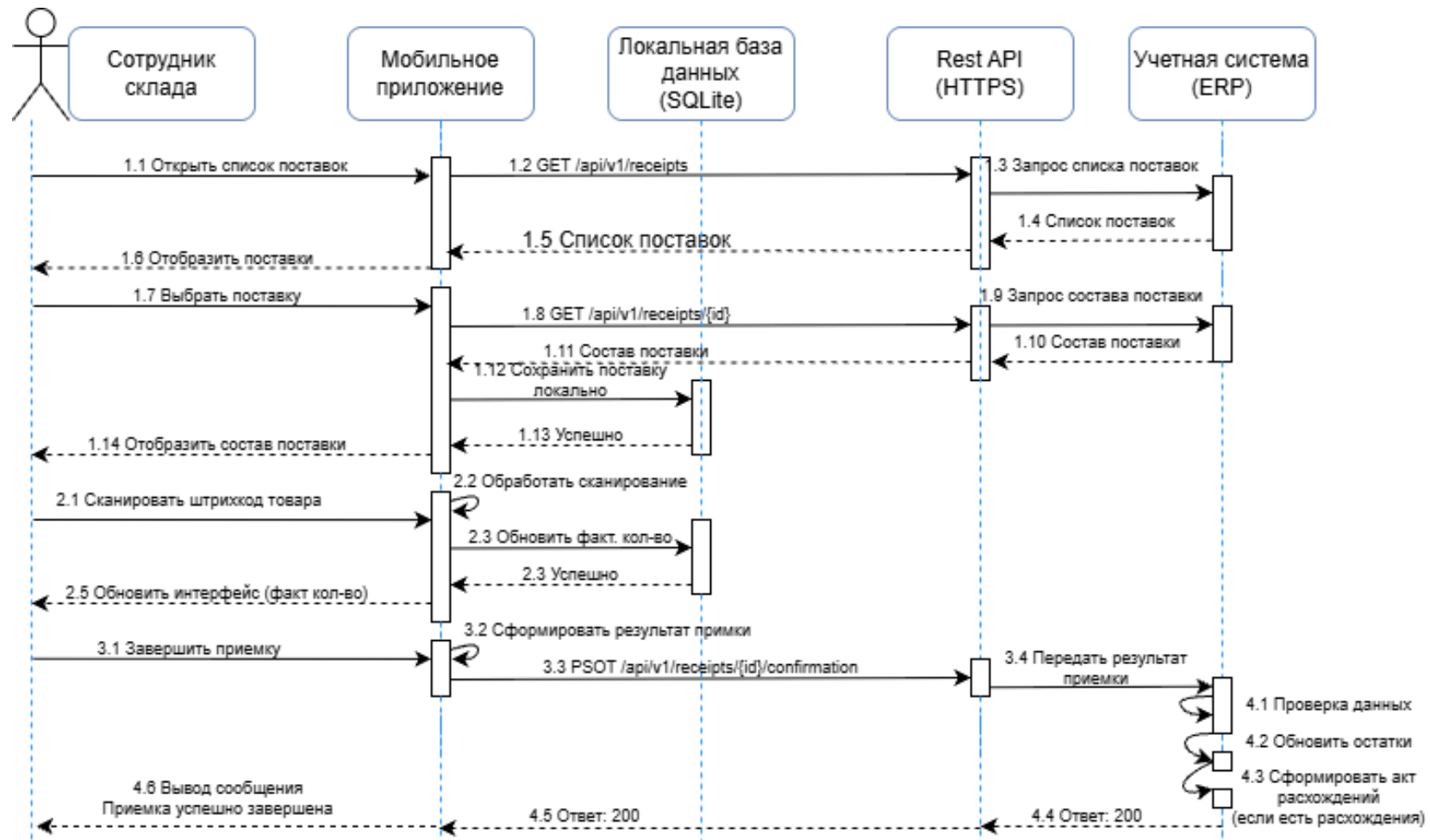
4. Интеграционный слой (RST API)

Взаимодействие между мобильным приложением и учетной системой через REST API.

- **Запросы приложения (Входящие)**
 - GET /api/v1/receipts – запрос списка доступных документов поступления.
 - GET /api/v1/receipts/{v1} – запрос детального состава конкретной поставки для кеширования на ТСД.
- **Передача результатов (Исходящие)**
 - POST /v1/receipts/{id}/confirmation – отправка финального отчета о фактически принятом товаре для проведения документа и фиксации расхождений.

6. Sequence Diagram

Теперь можно построить диаграмму последовательностей для полного осознания работы нашей программы.



Ссылка на Репозиторий с данной схемой - <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Описание Sequence Diagram

Участники процесса

- **Сотрудник склада** - пользователь, совершающий физические действия.
- **Мобильное приложение** - фронтенд-клиент (интерфейс на ТСД).
- **Локальная база данных (SQLite)** - хранилище на мобильном устройстве для работы офлайн.
- **Rest API (HTTPS)** - интеграционный слой для обмена данными между клиентом и сервером.
- **Учетная система (ERP)** - бэкенд, центральная база данных организации.

Этап 1. Загрузка и инициализация поставки

- 1.7 – 1.4:** Сотрудник нажимает “Открыть список поставок”. Мобильное приложение отправляет запрос GET /api/v1/receipts через Rest API в ERP-систему.
- 1.5 – 1.6:** ERP возвращает список поставок, Rest API транслирует его в мобильное приложение, и оно отображает этот список сотруднику на экране.
- 1.8 – 1.10:** Сотрудник выбирает конкретную поставку. Приложение отправляет запрос состава этой поставки по ID (GET /api/v1/receipts/{id}) в ERP. ERP возвращает детальный состав товаров.
- 1.11 – 1.14:** Приложение не просто показывает данные, а выполняет команду **1.12** “Сохранить поставку локально” в базу данных **SQLite**. После успешного сохранения (**1.13 Успешно**) состав поставки отображается сотруднику склада (**1.14**).

Этап 2. Процесс сканирования товаров

- 2.1:** Сотрудник физически сканирует штрихкод товара.
- 2.2:** Мобильное приложение выполняет внутреннее действие “Обработать сканирование”.
- 2.3:** Приложение отправляет запрос “Обновить факт. кол-во” напрямую в **Локальную БД (SQLite)**. Внешние сетевые запросы к API/ERP на этом шаге отсутствуют.
- 2.5:** После подтверждения от локальной БД (**2.4**) приложение обновляет интерфейс, показывая сотруднику актуальное фактическое количество принятого товара.

Этап 3. Завершение приемки и синхронизация

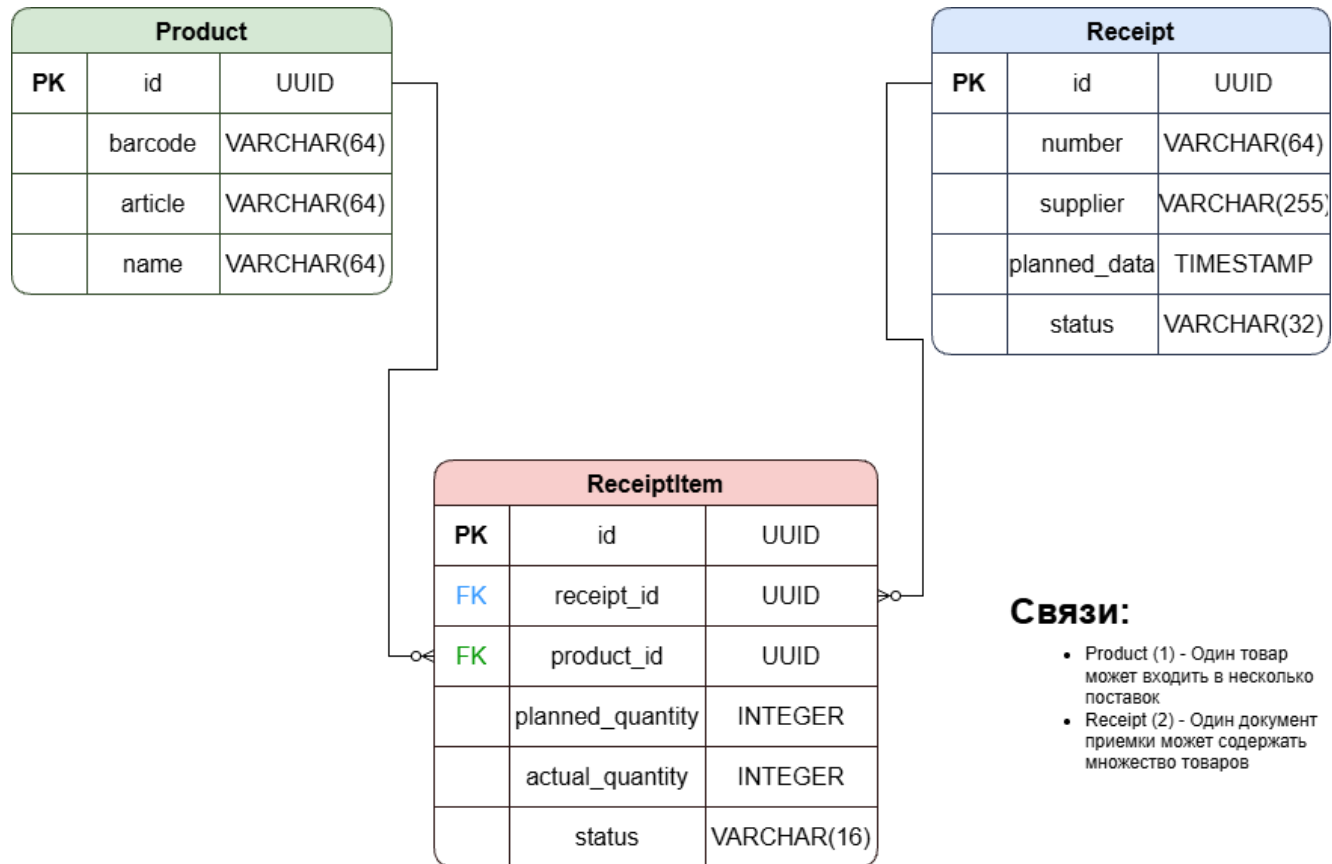
- 3.1 – 3.2:** Сотрудник нажимает кнопку “Завершить приемку”. Приложение выполняет внутреннюю сборку данных (“Сформировать результат приемки”).
- 3.3 – 3.4:** Приложение отправляет итоговый пакет данных в Rest API методом POST /api/v1/receipts/{id}/confirmation (примечание: на схеме опечатка в слове PSOT вместо POST). API передает результат приемки в ERP.
- 4.1 – 4.3 (Внутренняя логика ERP):** Учетная система последовательно выполняет три внутренних действия:
- Проверка данных.
 - Обновить остатки (на основном складе).

Сформировать акт расхождений (запускается автоматически, если факт не сошелся с планом).

4.4 – 4.6: ERP возвращает успешный статус ответа **4.4 Ответ: 200** через API в мобильное приложение. На экране сотрудника склада выводится финальное сообщение **“Приемка успешно завершена”**.

7. Модель данных

Теперь построим ER диаграмму:



Ссылка на Репозиторий с данной схемой - <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Описание сущностей (Таблиц)

1. Product - справочник номенклатурыid

(PK): Первичный ключ, уникальный идентификатор товара в формате UUID.

- **barcode:** Штрихкод товара (VARCHAR(64)), по которому мобильное приложение осуществляет поиск при сканировании.
- **article:** Артикул товара (VARCHAR(64)), текстовый код номенклатуры.
- **name:** Наименование товара (VARCHAR(64)).
- **Receipt (Поставка / Документ приемки)** - заголовок документа

2. Receipt - справочник номенклатурыid

(PK): Первичный ключ, уникальный идентификатор документа (UUID).

- **number:** Номер документа поставки (VARCHAR(64)).
- **supplier:** Наименование поставщика (VARCHAR(255)).
- **planned_data:** Планируемая дата и время поставки/приемки (TIMESTAMP).
- **status:** Текущий статус всего документа, например: “Новый”, “В работе”, “Завершен” (VARCHAR(32)).

3. ReceiptItem - промежуточная таблицаid

(PK): Собственный первичный ключ строки (UUID).

- **receipt_id (FK):** Внешний ключ, указывающий на конкретный документ из таблицы Receipt.
- **product_id (FK):** Внешний ключ, указывающий на конкретный товар из таблицы Product.
- **planned_quantity:** Плановое количество товара, которое ожидается от поставщика (INTEGER).
- **actual_quantity:** Фактическое количество товара, которое сотрудник склада подтвердил сканированием (INTEGER).
- **status:** Статус обработки конкретной строки, например: “Не принято”, “Принято полностью”, “Расхождение” (VARCHAR(16)).

8. Rest API

REST API обеспечивает взаимодействие между учетной системой и мобильным приложением.

Через API мобильное приложение получает документы поставки и передает результаты приемки обратно в учетную систему.

Обмен данными осуществляется по протоколу **HTTPS** в формате **JSON**.

Метод 1. Получение списка поставок

Получение списка документов, доступных для приемки.

Запрос

GET /api/v1/receipts

Ответ

```
[
  {
    "id": 101,
    "number": "PR-000101",
    "supplier": "ООО Ромашка",
    "plannedDate": "2025-07-20",
    "status": "NEW"
  },
  {
    "id": 102,
    "number": "PR-000102",
    "supplier": "ООО Продукт",
    "plannedDate": "2025-07-20",
    "status": "IN_PROGRESS"
  }
]
```

Метод 2. Получение состава поставки

Возвращает список товаров выбранного документа.

Запрос

GET /api/v1/receipts/{id}

Например

GET /api/v1/receipts/101

Ответ

```
{
  "id": 101,
  "number": "PR-000101",
  "supplier": "ООО Ромашка",
  "items": [
    {
      "productId": 1,
      "barcode": "4601234567895",
      "name": "Молоко 2.5%",
      "plannedQuantity": 15,
      "actualQuantity": 0
    },
    {

```

```
        "productId": 2,  
        "barcode": "4605555555555",  
        "name": "Кефир 1%",  
        "plannedQuantity": 8,  
        "actualQuantity": 0  
    }  
]  
}
```

Метод 3. Передача результатов приемки

После завершения приемки мобильное приложение отправляет фактически принятое количество товаров.

Запрос

POST /api/v1/receipts/{id}/confirm

Тело запроса

```
{  
  "receiptId": 101,  
  "items": [  
    {  
      "productId": 1,  
      "actualQuantity": 15  
    },  
    {  
      "productId": 2,  
      "actualQuantity": 6  
    }  
  ]  
}
```

Ответ

```
{  
  "status": "SUCCESS",  
  "message": "Приемка успешно завершена.",  
  "discrepanciesCreated": true  
}
```


Последовательность взаимодействия

Шаг	Мобильное приложение	REST API	Учетная система
1	Запрашивает список поставок	GET /receipts	Возвращает документы
2	Открывает поставку	GET /receipts/{id}	Возвращает список товаров
3	Выполняет приемку	-	-
4	Передает результаты	POST /receipts/{id}/confirm	Обновляет остатки и формирует акт расхождений

Обработка ошибок

При взаимодействии API используются стандартные HTTP-коды ответов.

Код	Описание
200 OK	Запрос успешно обработан
400 Bad Request	Некорректные данные запроса
401 Unauthorized	Пользователь не авторизован
404 Not Found	Документ поставки не найден
500 Internal Server Error	Внутренняя ошибка учетной системы

9. Пользовательский интерфейс

В данном разделе представлен подробный разбор графического интерфейса пользователя (UI) мобильного приложения для приемки товаров.

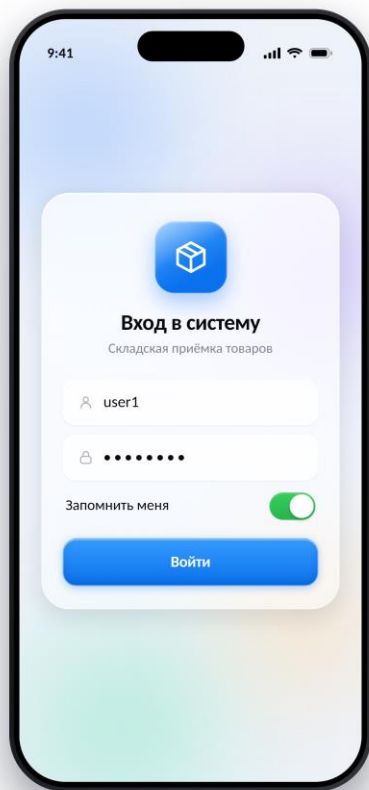
Цель этого блока — продемонстрировать, как бизнес-логика (описанная в BPMN) и технические сценарии (зафиксированные в Sequence-диаграмме) воплощаются в реальных экранах, с которыми взаимодействует сотрудник склада.

Для каждого окна системы приведено описание:

- **Фронтенд-логики:** как интерфейс реагирует на действия пользователя, визуализирует статусы и работает с локальными данными на устройстве (Offline-First).
- **Бэкенд-взаимодействия:** какие системные процессы и API-запросы запускаются на сервере при переходе между экранами.
- Этот разбор служит связующим звеном между проектными схемами и этапом разработки, обеспечивая однозначное понимание того, как должна работать готовая программа.

Ссылка на репозиторий и видео работы UI – <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

1. Авторизация

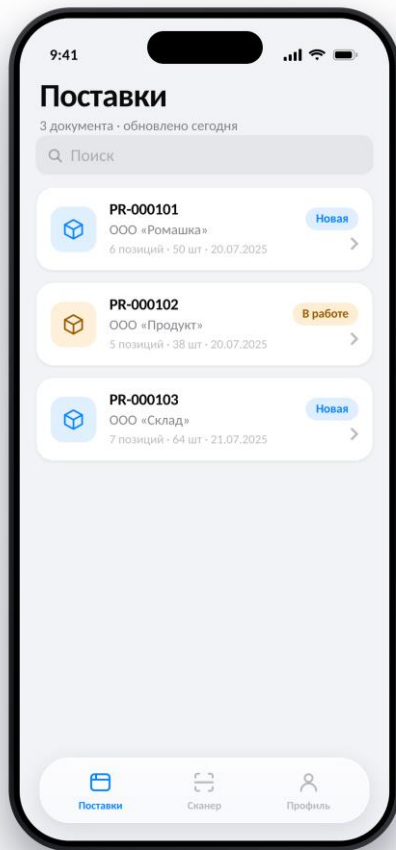


Ссылка на репозиторий - <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Этот интерфейс запускает самый первый этап ранее описанного бизнес-процесса и диаграммы последовательности (Sequence diagram).

- На фронте
 - Сбор UI данных - пользователь вводит логин и пароль
 - Локальное состояние - Переключатель “Запомнить меня”
 - Инициализация запроса - Кнопка “Войти” отправляет данные в сторону API
- На Бэке
 - Аутентификация - Сервер принимает запрос, Хэширует пароль и проверяет его в БД юзеров
 - Генерация сессии – Бэк создает “JWT” токен
 - Определение прав – Система проверяет роль по токену
 - Ответ системы – Сервер возвращает 200 – Ок и список доступных операций для юзера

2. Поставки

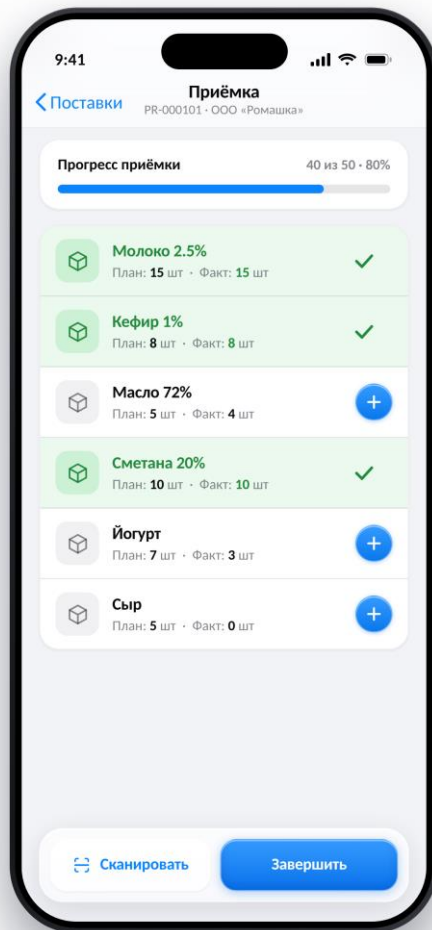


Ссылка на репозиторий – <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Этот интерфейс отображает данные из сущности Receipt нашей ER диаграммы

- Фронт
 - Отображает данные – Выводит список документов в виде карточек со статусом (“Новая”, “В работе”). Данные берутся из таблицы Receipt
 - Локальная фильтрация – поле “Поиск”
 - Инициализация выбора – Нажатие на карточку запускает “Выбрать поставку”
- Бэк
 - Выгрузка документов – По запросу GET /api/v1/receipts бэк агрегирует из бд все активные на сегодня документы приемки.
 - Расчет строк – Для каждой карточки сервер рассчитывает или отдает из метаданных значения из таблицы ReceiptItem

3. Приемка



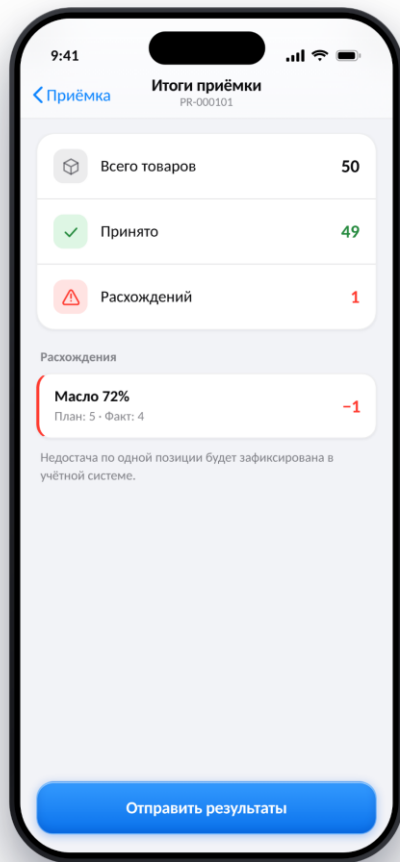
Ссылка на репозиторий – <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Этот интерфейс визуализирует состав конкретной поставки и взаимодействует с таблицей ReceiptItem из ER диаграммы.

- Фронт
 - Отображение состава – выводит список позиций, где для каждой строки из БД подтягивается наименование товара (Product.name) план (ReceiptItems.planne_quantity) и факт (ReceiptItems.actual_quantity)
 - Цветовая индикация статусов: Зеленый фон с галочкой – Позиции где actual_quantity == planned_quantity. Строка получает статус “Принято полностью”. Серый фон – Позиции которые не обработаны или приняты частично (actual_quantity < planned_quantity)

- Расчет прогресс-бара – Приложение на лету вычисляет процент выполнения приемки (сумма actual_quantity делится на planned_quantity) и обновляет индикатор вверху экрана.
- Элементы управления – Кнопка “Сканирование” активирует Scanner Modul, а кнопка “Завершить” переводит к этапу синхронизации. Кнопки “+” позволяют вручную инкрементировать без сканера.
- Бэк
 - Офлайн изоляция – При инкременте кол-ва или сканировании штрихкода бэк не задействуется. Все изменения мгновенно идут в локальную БД “SQLite”
 - Ожидание отправки – Бэк на данном этапе находится в режиме ожидания и никак не меняет остатки на основном складе до тех пор пока юзер не нажмет на кнопку “Завершить” и приложение не пришлет итоговый POST.

4. Итог



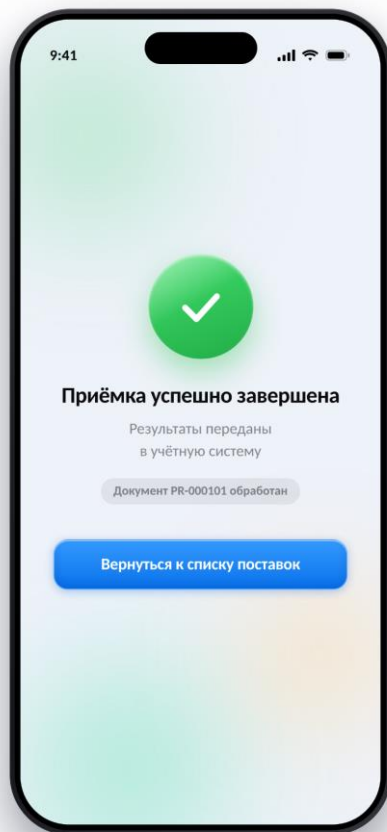
Ссылка на репозиторий – <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Это финальный интерфейс перед отправки данных в ERP систему.

- Фронт
 - Агрегация сводных данных – приложение обращается к локальной БД SQLite и рассчитывает три ключевые метрики.
 - Всего товаров – сумма `planned_quantity`
 - Принято – Сумма всех отскан. штук `actual_quantity`
 - Расхождение – Количество единиц товара, для которых `planned_quantity != actual_quantity`
 - Визуализация проблемных позиций – UI фильтрует строки и выводит “Расхождения”.
 - Инициализация отправки – Нажатие на кнопку “Отправить результат” кодирует данные поставки в один JSON и иницирует POST `/api/v1/receipts/{id}/confirmation`
- Бэк
 - Обработка расхождений – Сервер принимает пакет данных, так как обнаружена недостача “Есть расхождения? = Да”. Запускается два параллельных процесса: Обновление остатков

- Фактические принятые единицы товара приходят на баланс склада.
- Акт расхождений – Система автоматически генерирует (Акт расхождений)
- Заккрытие документа – Сервер меняет статус документа Receipt.status на “Завершен с расхождениями” и возвращает 200 – Ок.

5. Готово

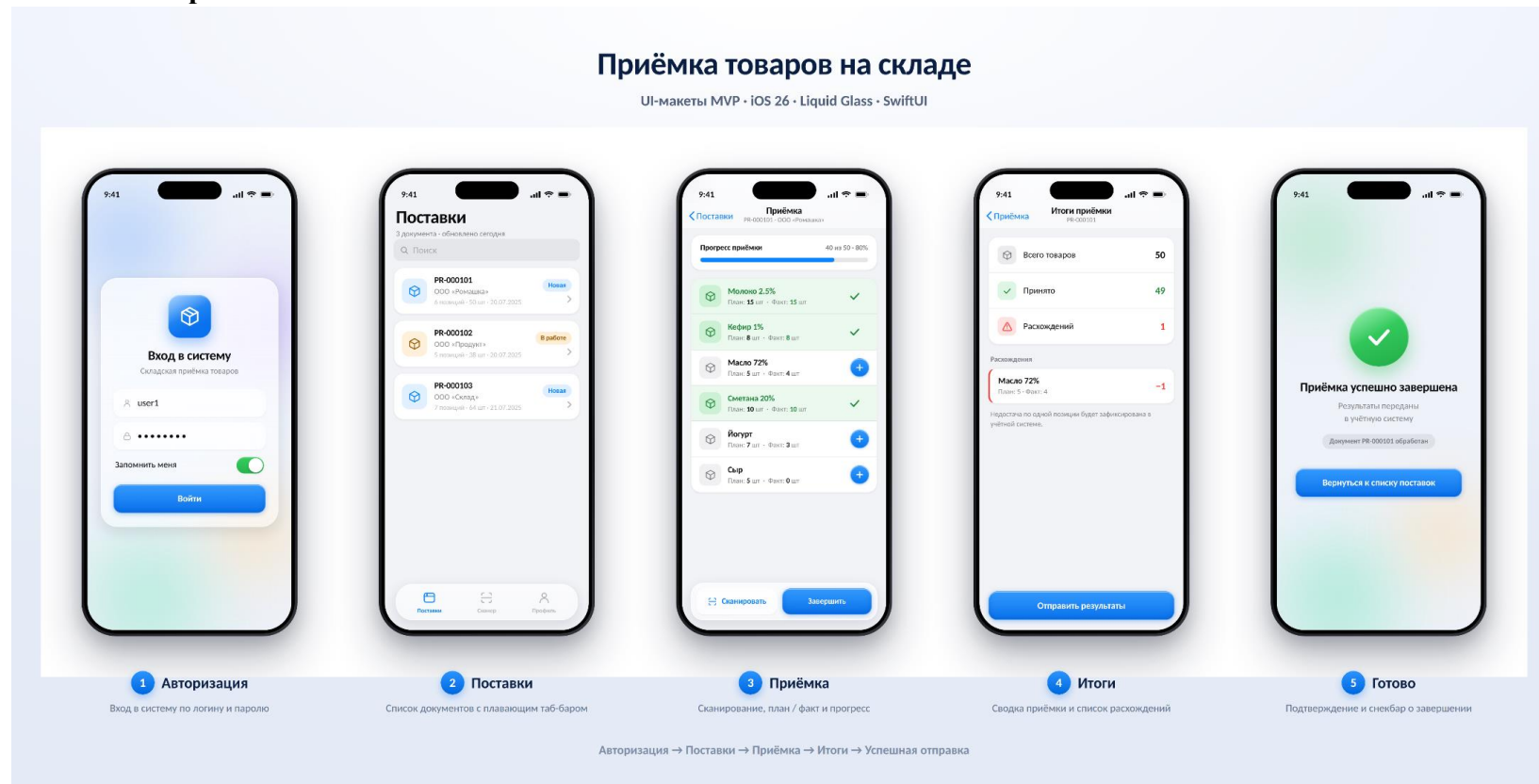


Ссылка на репозиторий – <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

Этот интерфейс подтверждает пользователю, что все данные синхронизированы, а транзакция на сервере успешно закрыта.

- Фронт
 - Информирование о статусе
 - Идентификация документа
 - Очистка локального кэша
 - Навигация
- Бэк
 - Финальная фиксация

6. Макет – обзор



Ссылка на репозиторий и видео работы UI – <https://github.com/Smatakov/Mobile-Scanner-UI-Diagramms>

10. Возможные улучшения

В качестве дальнейшего развития приложения рекомендуется внедрить модуль умной аналитики для прогнозирования времени приемки и автоматического выявления недобросовестных поставщиков.

Также ускорить работу сотрудников поможет интеграция с носимыми устройствами (голосовой ввод данных через гарнитуру и сканирование с помощью умных очков). Дополнительно стоит реализовать сквозную фотофиксацию брака, которая будет автоматически прикрепляться к формируемому в ERP акту расхождений.

11. Заключение

Разработанное проектное решение полностью описывает сквозной процесс мобильной приемки товаров — от авторизации пользователя до фиксации расхождений на бэкенде. Предложенная архитектура и детальный разбор UI-экранов обеспечивают прозрачную связь между действиями кладовщика и логикой ERP-системы. Реализация паттерна Offline-First гарантирует отказоустойчивость системы в условиях нестабильной связи на складе, а заложенная структура данных позволяет легко масштабировать функционал приложения в будущем.